

Digital Design

Counter

with Verilog Model

A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

**Some part of Slides Adopted from Digital System Book
by Frank Vahid**

Outline

- Counter
- Sync Counter
- Digital Clock : Case Study
- **Formal Model of Sequential Circuit**

Counter

- It simply count: 4 bit CTR: count 0000 to 1111 & repeat
- Other optional functions and variations
 - Start count at Specific point (say from 5 : 0101)
 - Stop count at Specific point (Say at 9: 1001)
 - Count only even numbers: 0, 2, 4, ..14, 0, 2
 - Count only odd numbers: 1, 3, 5,..15, 1
 - Count specific num in specific order & repeat:

1, 9, 4, 6, 7, 8. 3, 1, 9 ,4,...

Mostly used 4 bit counter

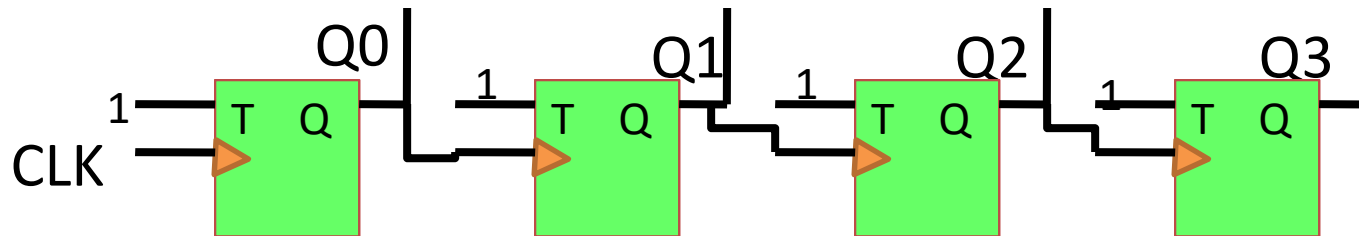
- Example of 4 bit counter
- Count from 0000 to 1111 and repeat
- Up counter : 0000 to 1111
- Down counter : 1111 to 0000
- Mod N counter:
 - Mod 10 counter : 0000 to 1001 (0 to 9) and repeat (Decimal Number)
 - Mod 6 counter : 000 to 101 (0 to 5) and repeat (Digital Clock 60 second, 60 minutes, 12)

How to design a simple counter

- Q0 change every time
- Q1 change in two time
- Q2 change in every four time
- Q3 changes in every eight time
- Q0, Q1 and Q2 changes can be modeled
 - Q0 can be modeled using T FF
 - Q1 : in term of Q0 and T FF
 - Q2 : in term of Q1 and T FF...

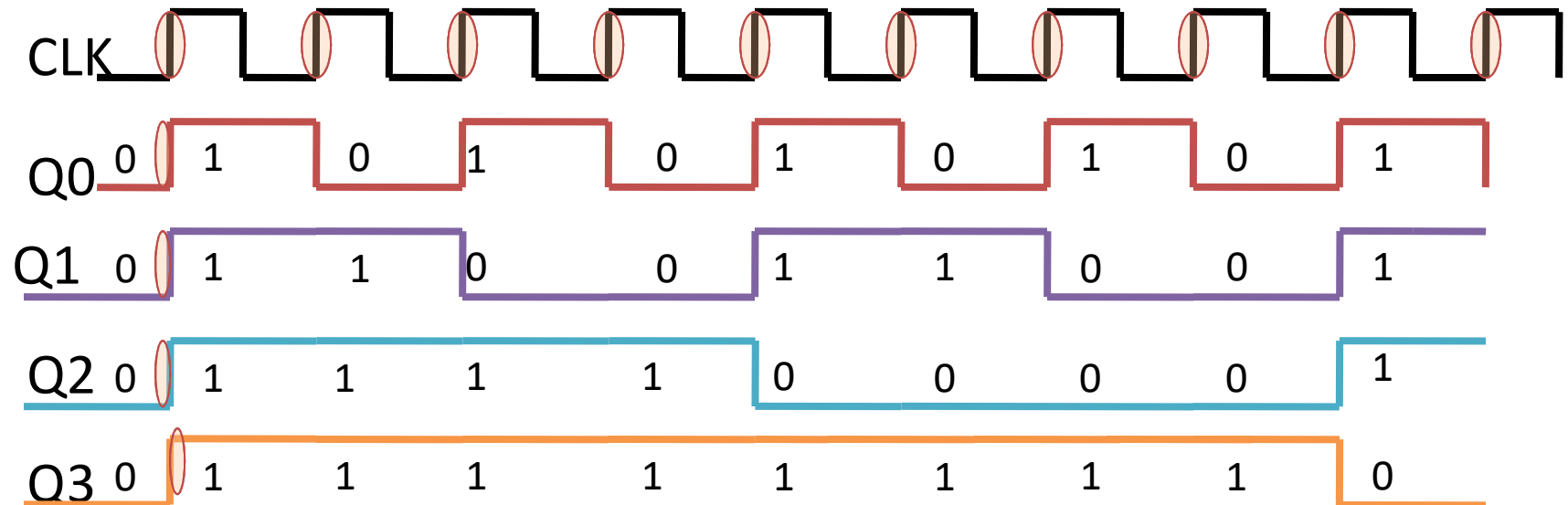
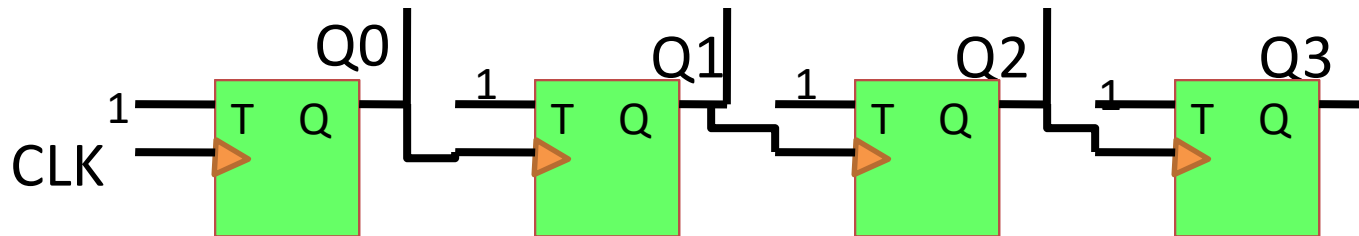
Q3	Q2	Q1	Q0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1

Binary Counter: using T FF

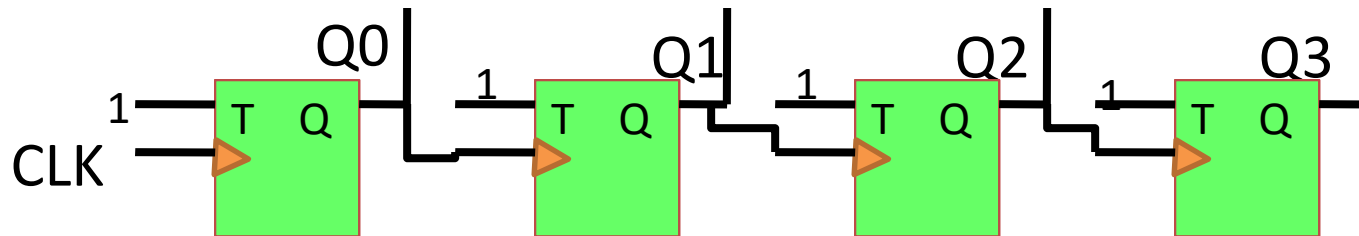


Does this circuit solve
our purpose ?

Binary Counter: using T FF

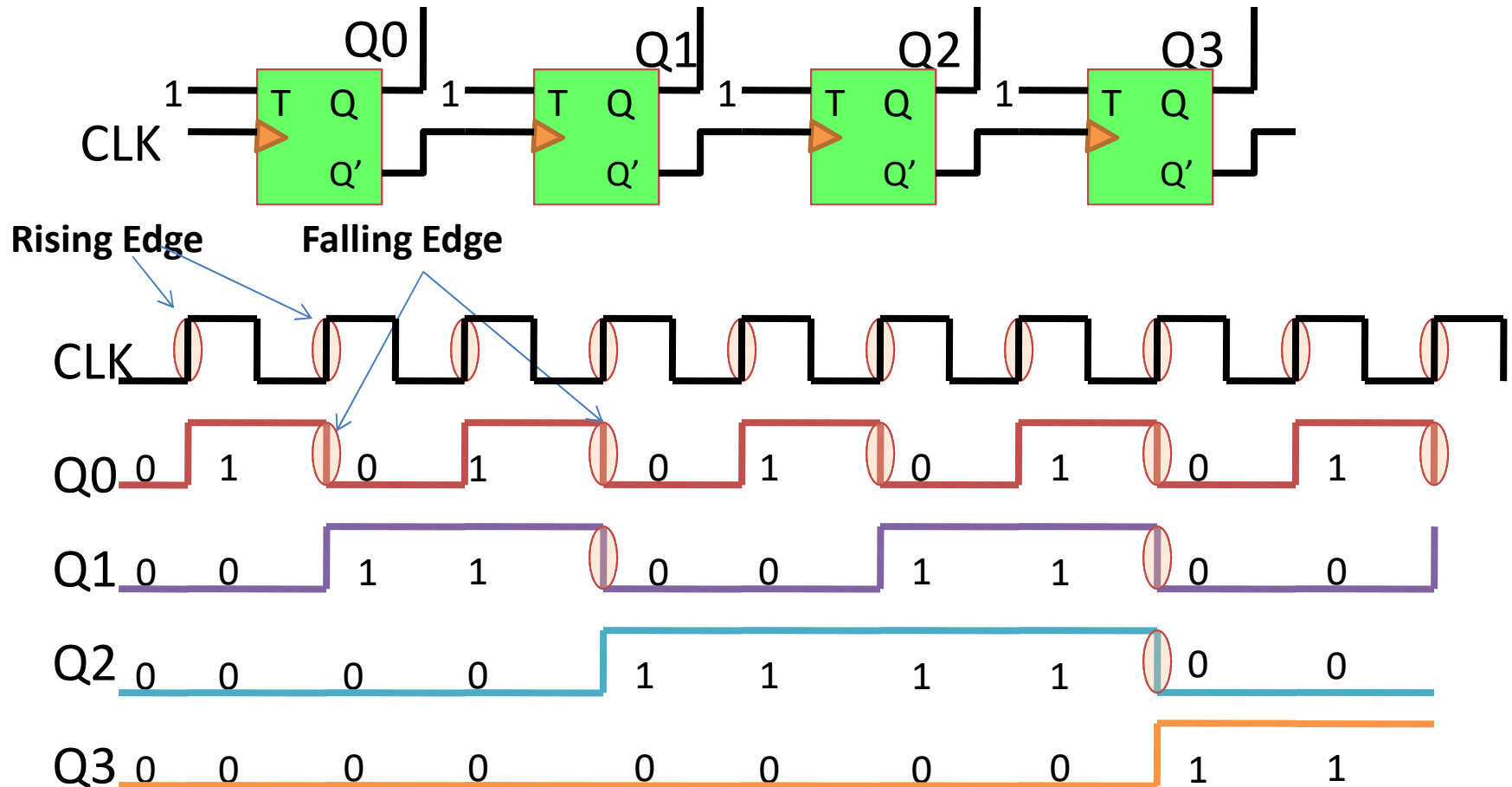


Binary Counter: using T FF



What is the problem
with this?

Binary Counter: using T FF



Verilog Code for 4 Bit Counter: T-FF

```
module tff (output reg q, input t,  
            input clk, input rst);  
    always @ (posedge rst) begin  
        if (rst) q=0;  
    end  
    always @ (posedge clk) begin  
        if (~rst) begin  
            if (t) q <= ~q; else q <= q;  
        end  
    end  
endmodule
```

Verilog Code for 4 Bit Counter

```
module ripplecounter(clk,t, rst, q);  
  input clk,t, rst;  
  output [3:0]q;  
  // initiate 4 T-FF to update the count  
  tff    tf1(q[0],t, clk, rst);  
  tff    tf2(q[1],t, ~q[0], rst);  
  tff    tf3(q[2],t, ~q[1], rst);  
  tff    tf4(q[3],t, ~q[2], rst);  
  
endmodule
```

Verilog Code for 4 Bit Counter: Test bench

```
module tb;  
reg clk, rst;  
reg t;  
wire [3:0] q;  
ripplecounter dut(clk,t,rst,q);
```

```
//Part I
```

```
//Part II
```

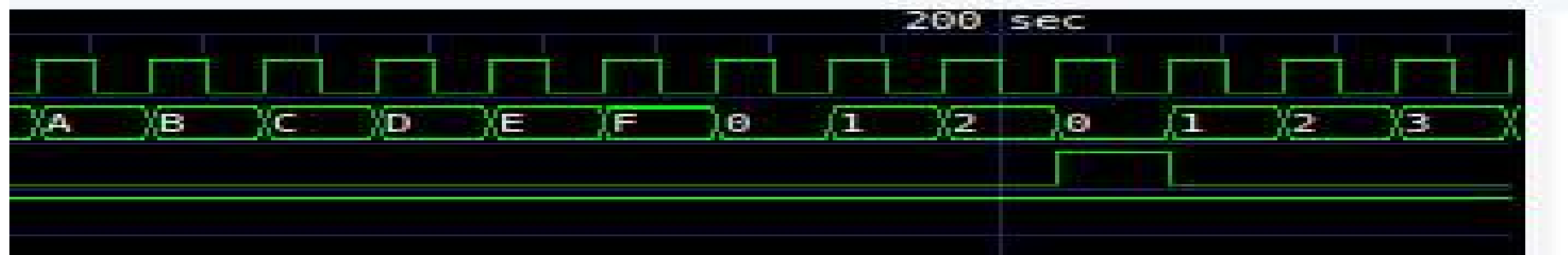
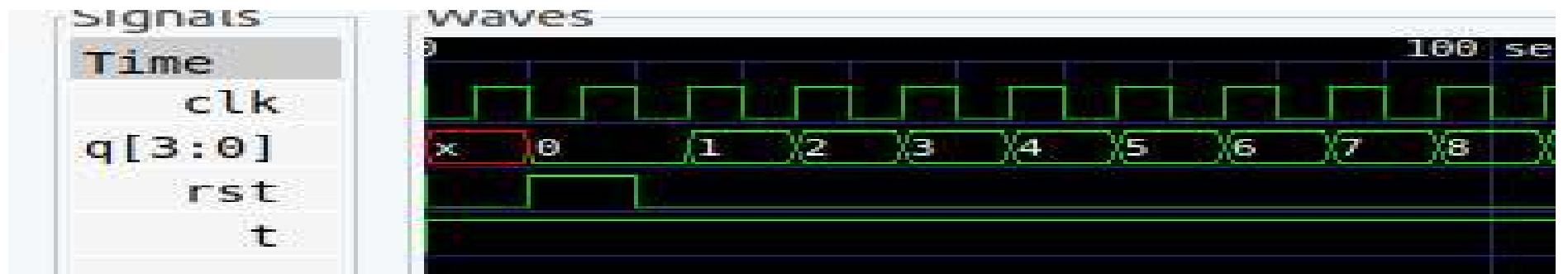
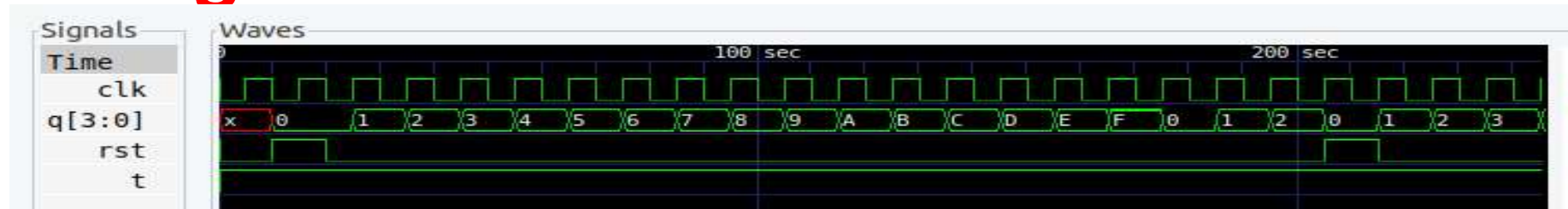
```
initial  
    $monitor("time=%g, rst=%b,  
             clk=%b,t=%b, q=%d",  
             $time,rst, clk,t, q);
```

```
endmodule
```

```
initial  
    clk = 0;  
always  
    #5 clk = ~clk;
```

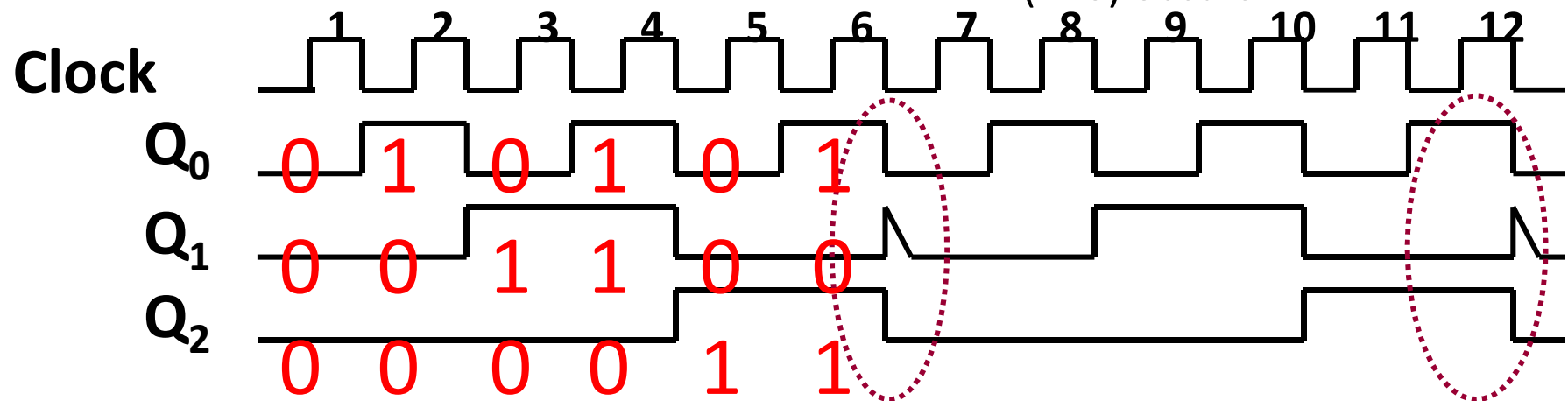
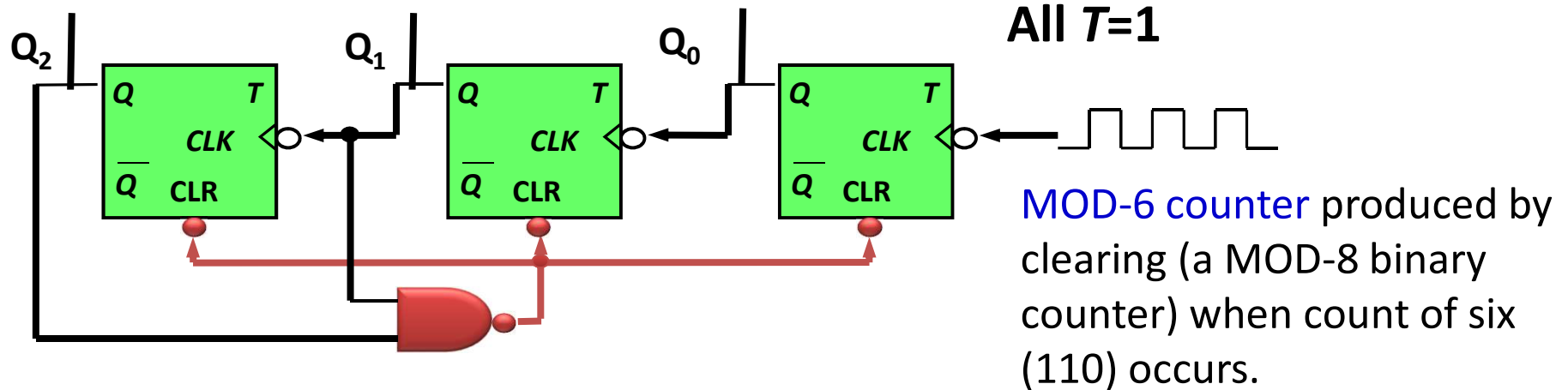
```
initial begin  
    $dumpfile("RCounter.vcd");  
    $dumpvars;  
    rst=0; t = 1;  
    #10 rst=1;  
    #10 rst=0;  
    #185 rst=1;  
    #10 rst=0;  
    #30 $finish;  
end
```

Verilog Code for 4 Bit Counter: WaveForm



Asyn. Counters with MOD no. $< 2^n$

Asyn. Counters with MOD 6



Verilog Code for Async Mod 6 Counter

```
module mod6ripplecounter(clk,t, q,rst, ffrst);  
  input clk,t, rst;  
  output [2:0]q;  
  output ffrst;  
  // initiate 3 T-FF to update the count  
  tff tf1(q[0],t, clk, ffrst);  
  tff tf2(q[1],t, ~q[0], ffrst);  
  tff tf3(q[2],t, ~q[1], ffrst);  
  assign #1 ffrst=rst|(q[2]&q[1]);  
endmodule
```

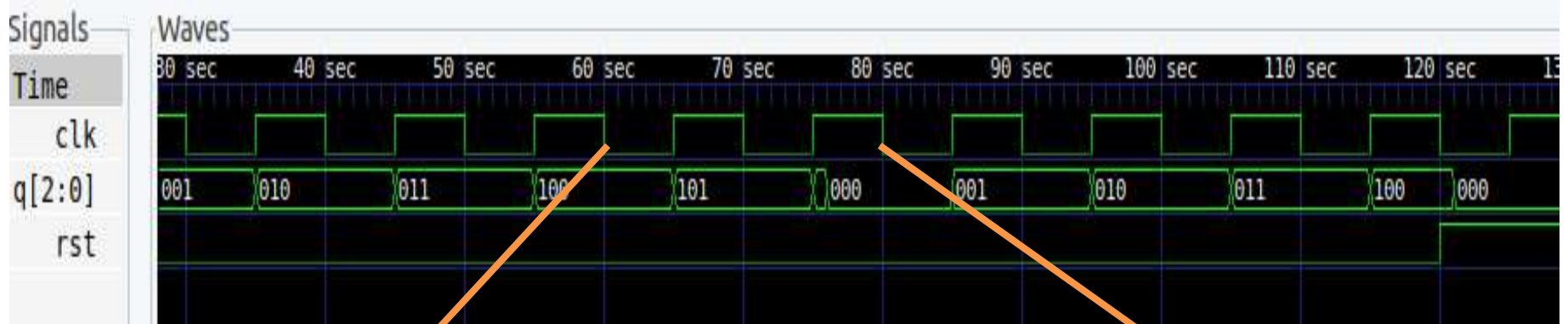

Verilog Code for Async Mod 6 Counter

```
module tb;  
  reg clk, rst, t;  
  wire ffrst, [2:0]q;  
  mod6ripplecounter  
    dut(clk,t,q, rst, ffrst);  
  //Part A  
  //Part B  
  initial  
    $monitor("time=%g, rst=%b,  
      clk=%b,t=%b, q=%d ffrst=%b"  
      , $time,rst, clk,t, q, ffrst);  
endmodule
```

```
initial  
  clk = 0;  
  always #5 clk = ~clk;
```

```
initial begin  
  $dumpfile("Mod6Counter.vcd");  
  $dumpvars;  
  rst=0; t = 1;  
  #10 rst=1; #10 rst=0;  
  #100 rst=1; #10 rst=0; #30  
  $finish;  
end
```

Waveform from GTKWAVE: Mod6 Ctr

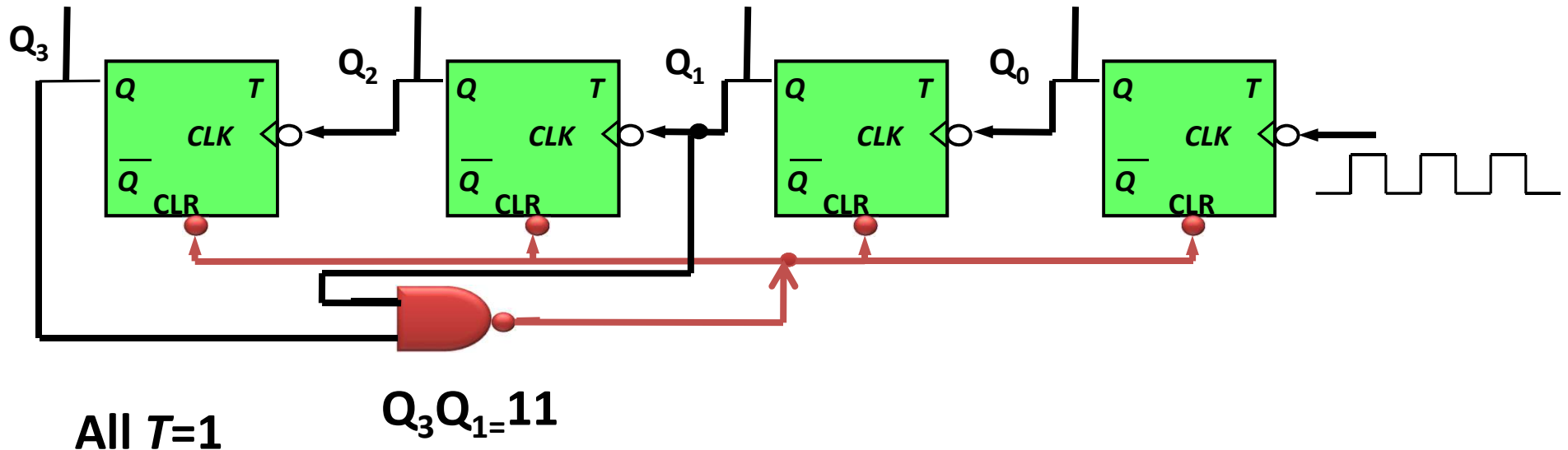


Zoomed view: 60s to 80s



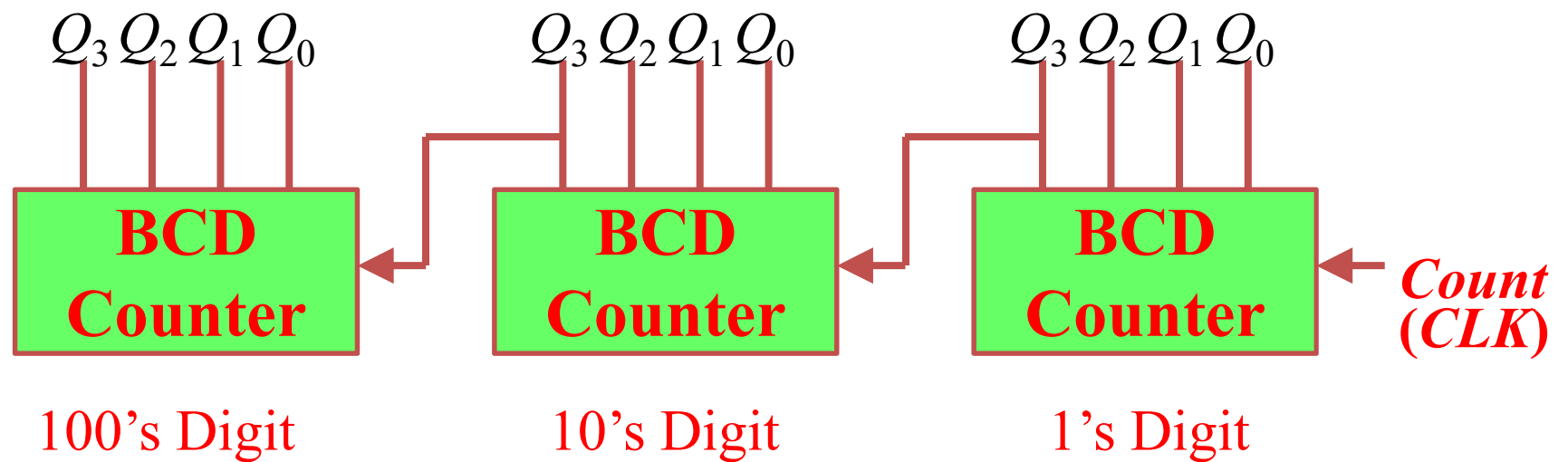
For 1 sec counter became 6 and get reset to zero

Asyn. Counters with MOD 10: BCD Ctr

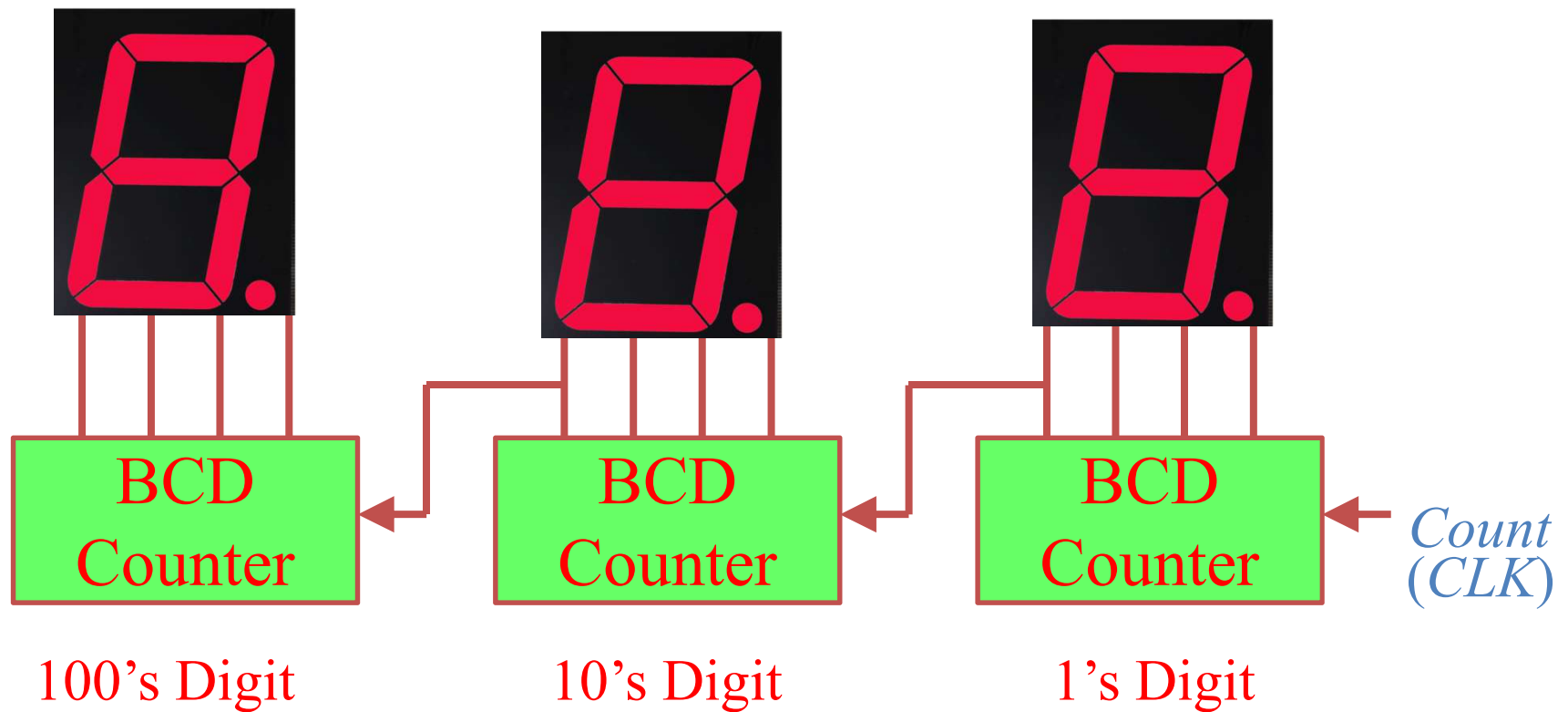


MOD-10 counter produced by clearing (a MOD-16 binary counter) when count of six (1010) occurs.

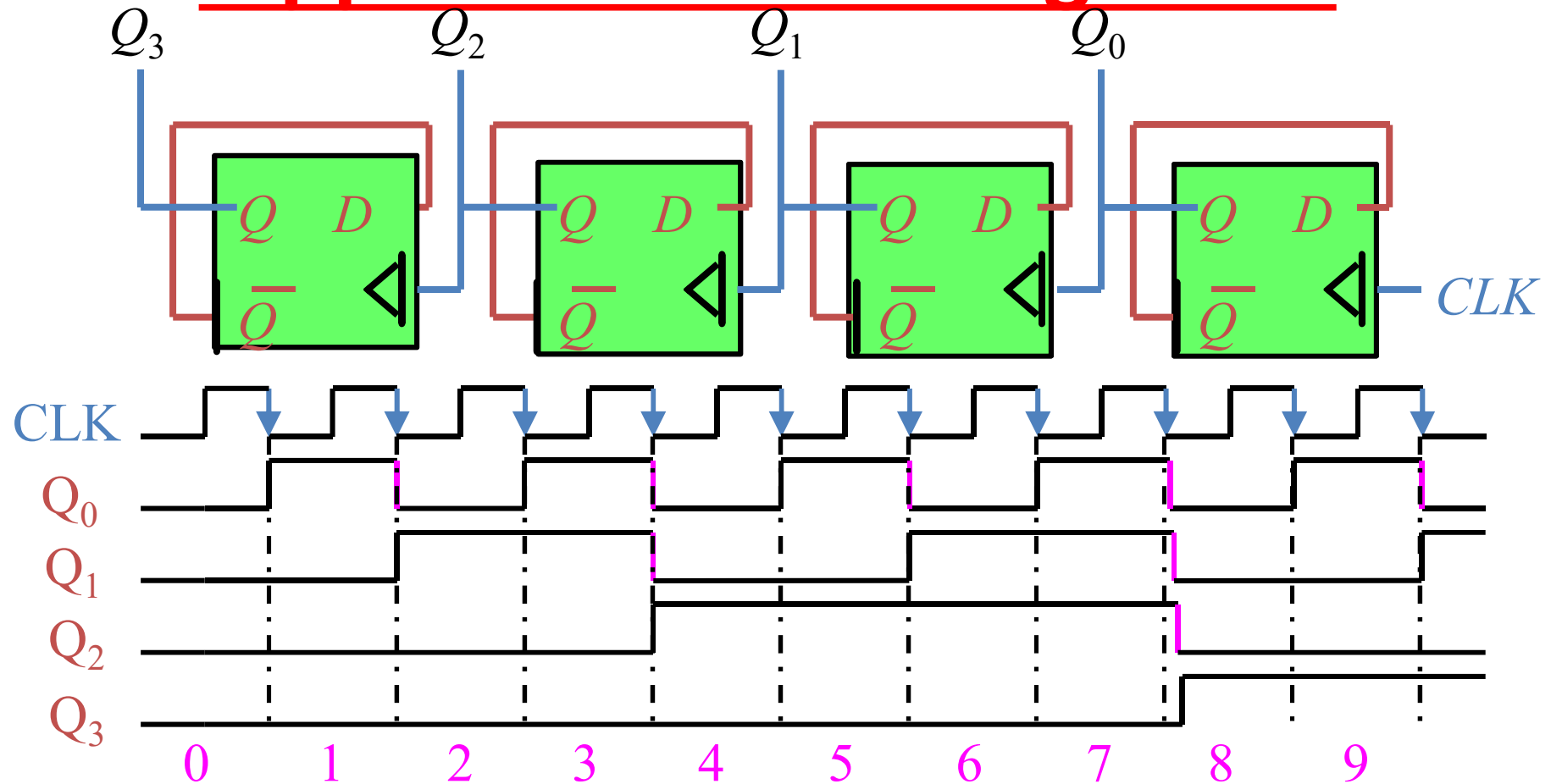
Decade Counter



Decade Counter: Interfaced BCDto7Seg Decoder +7Seg



Ripple Counters using D-FFs



Binary Counter: using T FF

- It is very easy to design Ripple counter

Is there any Issue
with this ?
If yes, what?

Binary Counter: using T FF

- It is very easy to design Ripple counter

First thing: Don't play
with clock but make
change to the Circuit..

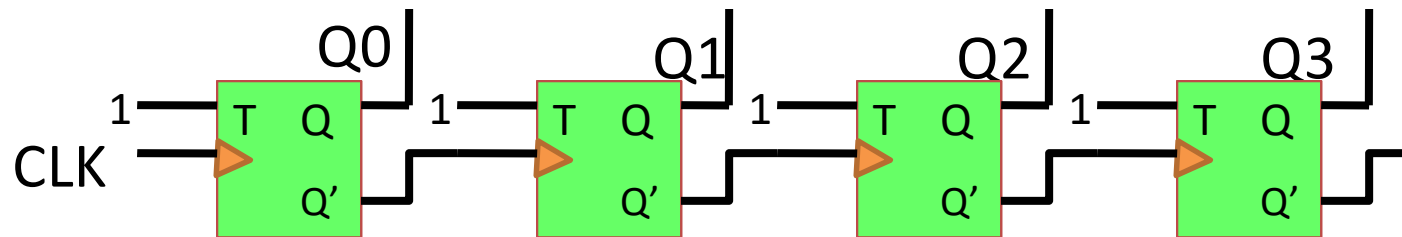
Problem with Ripple Counter

- FFs are not synchronized 😊 😞
- Synchronized “Wall Clocks” of Rail Station
 - Railway clocks : Synched Clock with Network time
 - NTP : network time protocol
- You can Sync clock of “Smart Mobile” with Network Clock which is in Sync with India Clock and World Clock

Problem with Ripple Counter

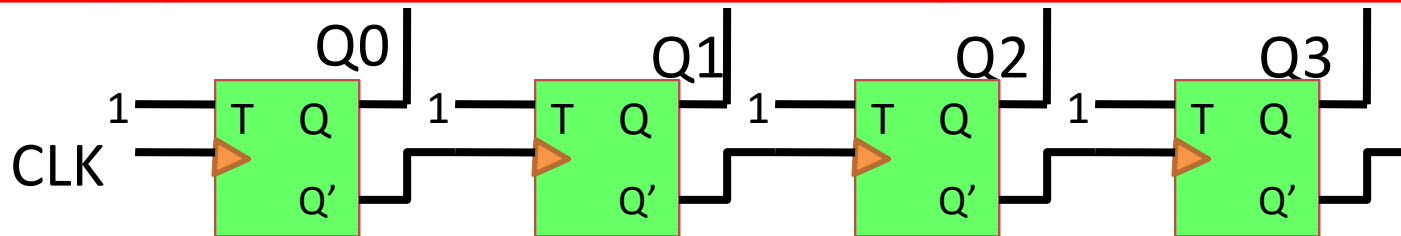
- Student A copies Solution from B with some error, C copies from B with some error and continues..
 - What will be the solution of Z. ($=Sol+26*error$)
 - Communication Gap
- So, all the FFs should take same clock signal

Ripple Counter : Asynchronous



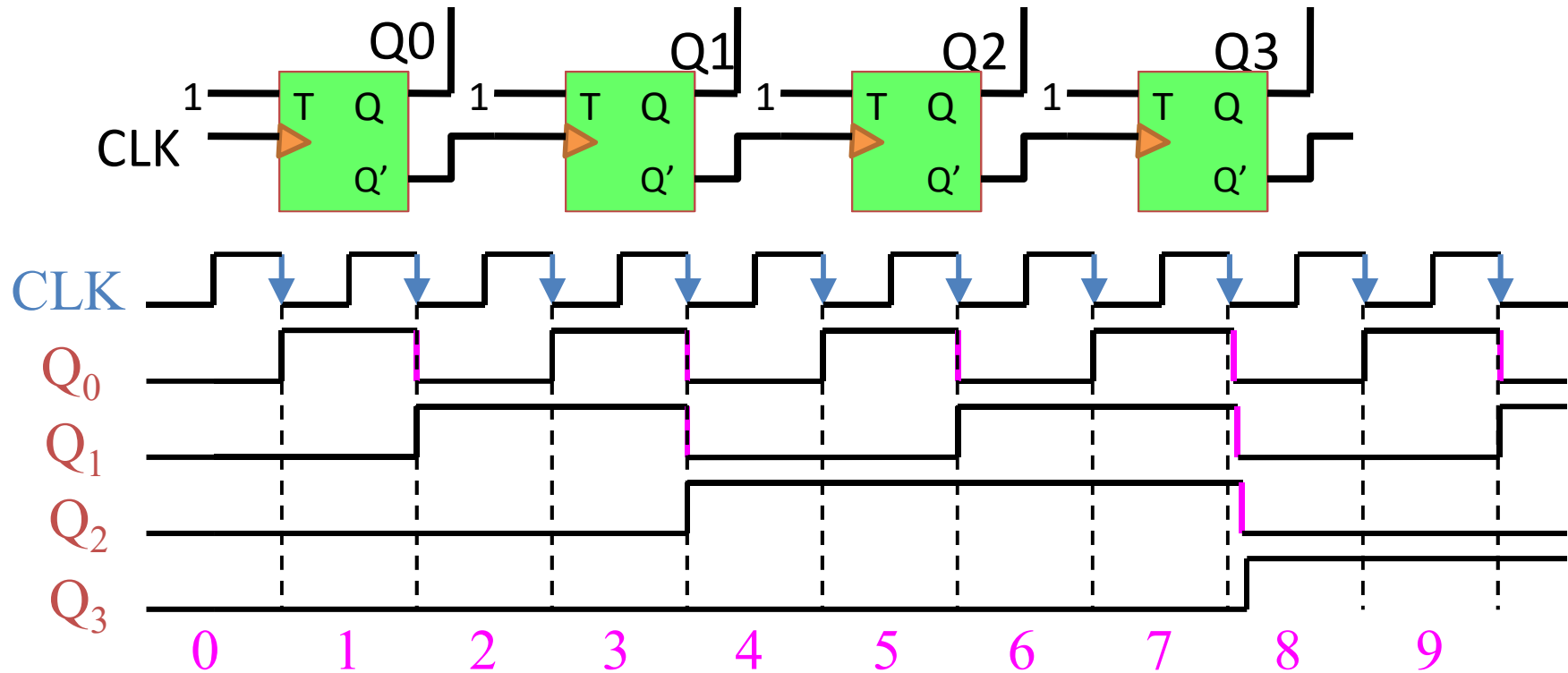
- Clock is applied at FF0, it propagate through to FF_n
- Change in State of Q_{i-1} is used to Toggle Q_i
- Input Clock to FF1= Skewed version of Clk of FF0
 - Clock + Propagation delay of FF

Ripple Counter : Asynchronous



- Rippling : Overall time delay of occurrence of count pulse and when stabilized count appear at O/P
- When counter : 1111..11 to 0000..00, toggle signal must propagate through all FFs
- **Worst case Settling time: $n \times t_{pd}$**
where t_{pd} = Propagation delay of a FF

Ripple Counter : Asynchronous



Code for 4 Bit Counter: T-FF with PD

```
module tff (output reg q, input t,  
            input clk, input rst);  
    always @ (posedge rst) begin  
        if (rst) q=0;  
    end  
    always @ (posedge clk) begin  
        if (~rst) begin #1; //Adding Propagation delay  
            if (t) q <= ~q; else q <= q;  
        end  
    end  
endmodule
```

Synchronous Counter

- **Synchronous Counter:** One single clock to all FFs
- Need to design and add Extra Circuitry to make it synchronous

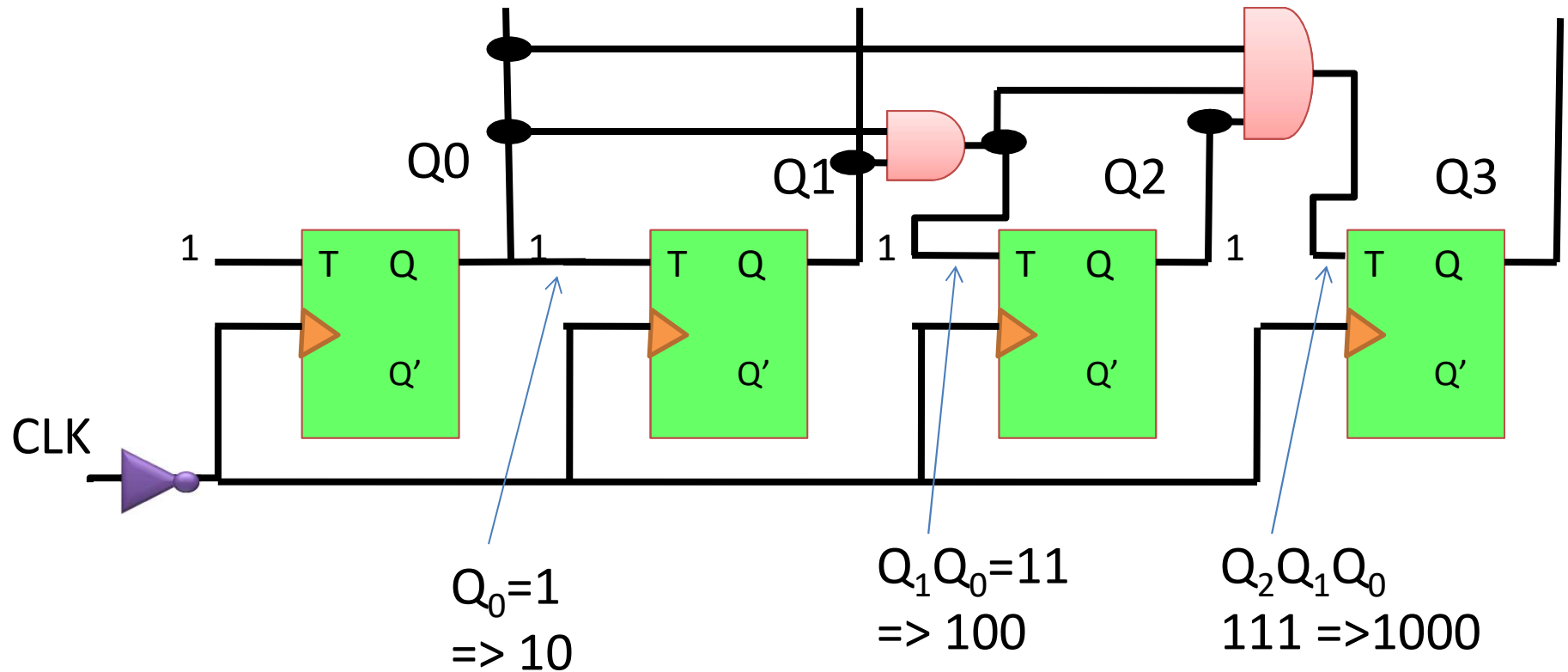
How to Design and put
extra circuitry?

How to design a simple counter

- Q0 change every time
- Q1 change when
 - Q0=1
- Q2 changes when
 - Q0=1 and Q1=1
- Q3 changes when
 - Q0=1 and Q1=1 and Q2=1

Q3	Q2	Q1	Q0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1

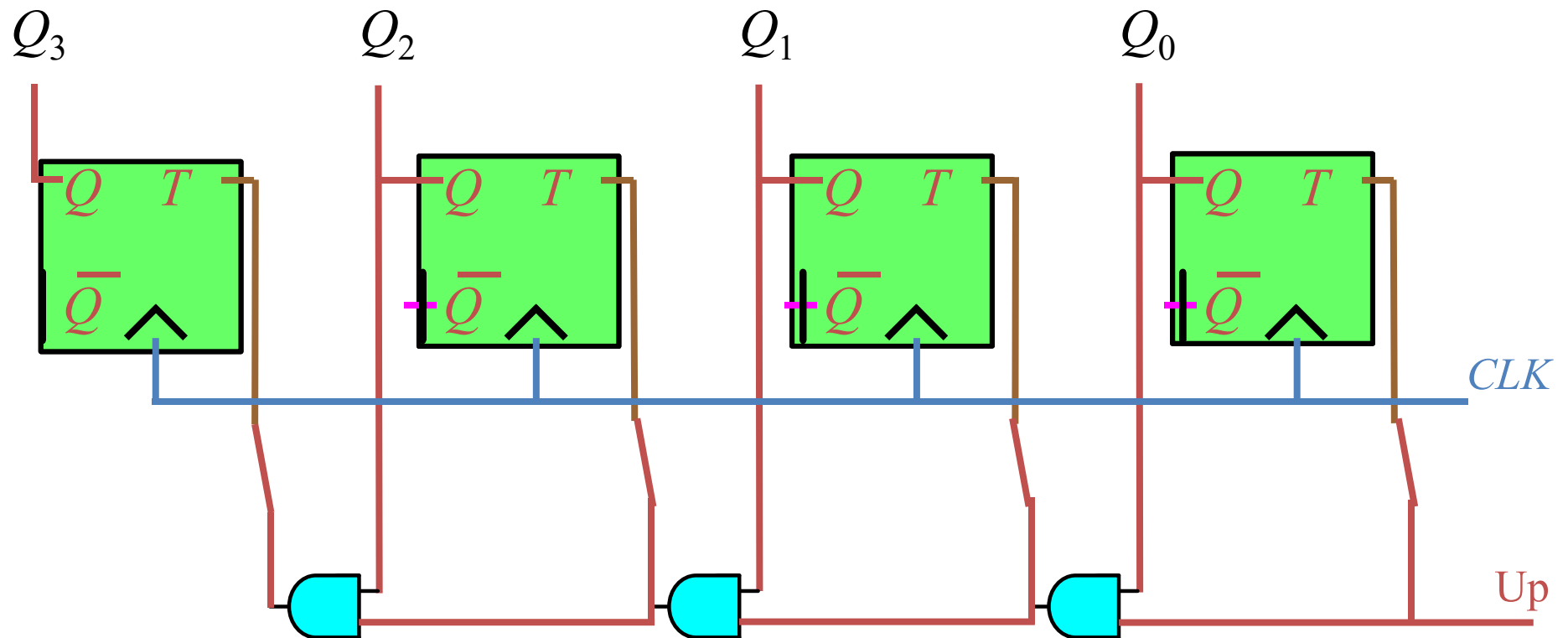
Binary Counter: Synchronous



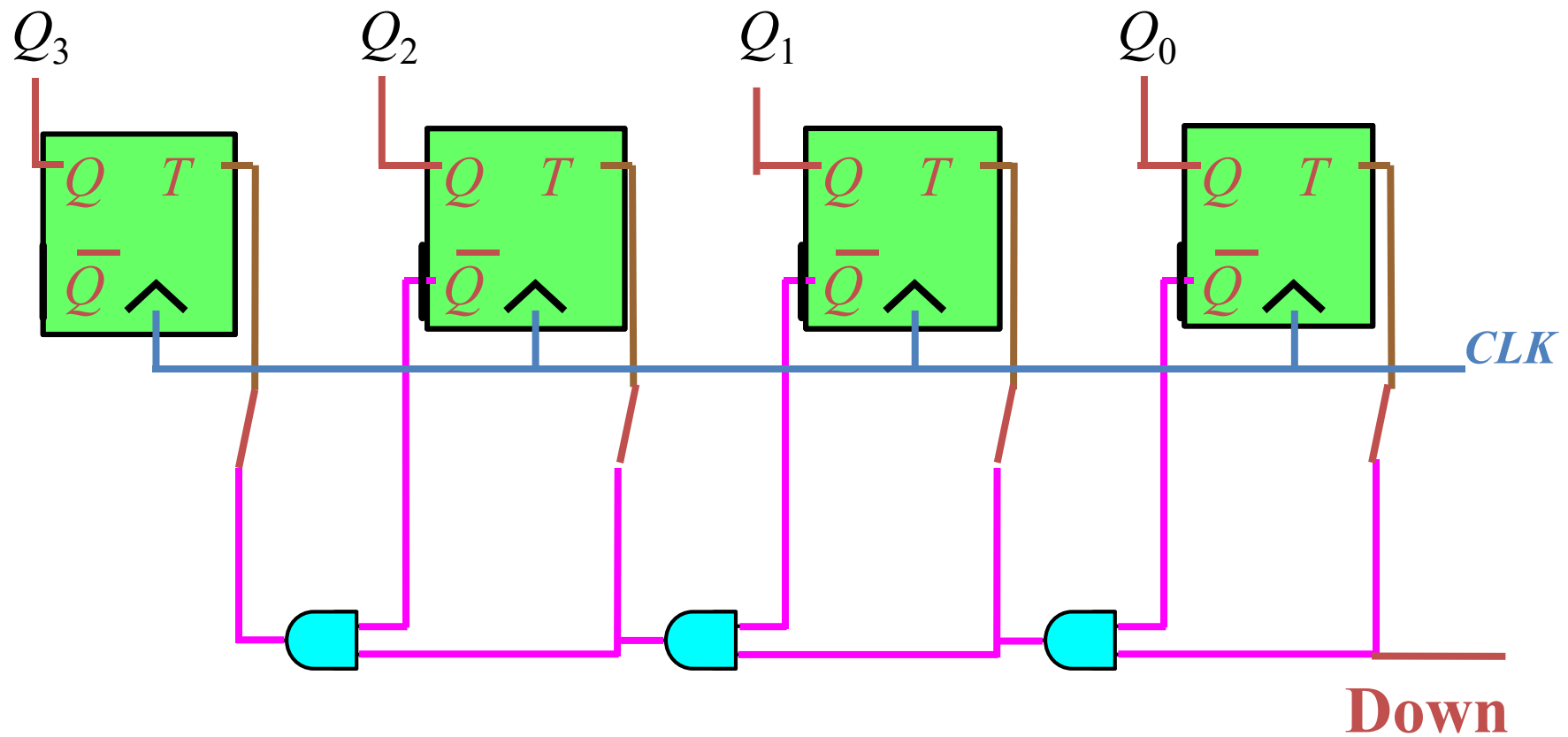
Verilog Code for 4 Bit Sync Counter

```
module ripplecounter(clk,t, rst, q);  
  input clk,t, rst;  
  output [3:0]q;  
  
  // initiate 4 T-FF to update the count  
  tff tf1(q[0],t, clk, rst);  
  tff tf2(q[1],q[0], clk, rst);  
  tff tf3(q[2],q[0]&q[1], clk, rst);  
  tff tf4(q[3],q[0]&q[1]&q[2], clk, rst);  
  
endmodule
```

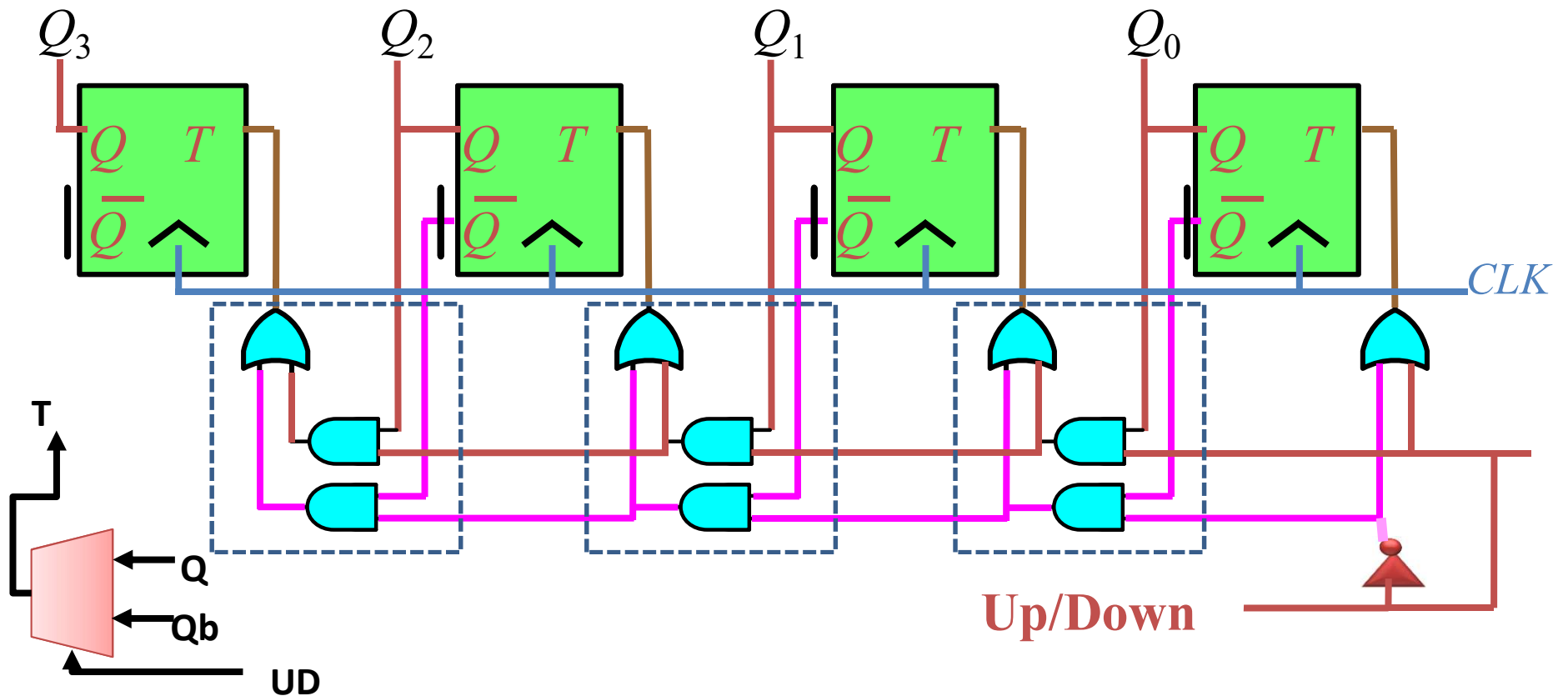
Sync: Binary UP Counter



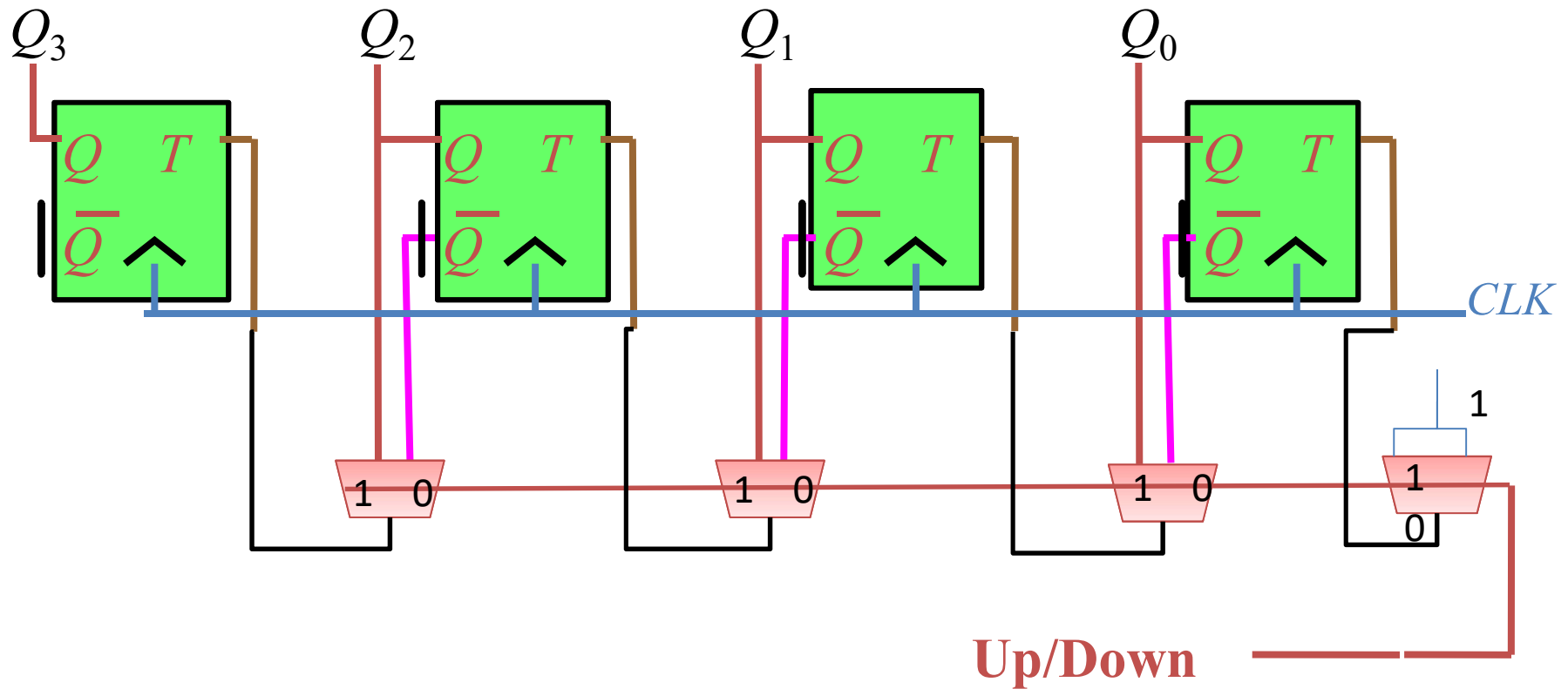
Sync: Binary Down Counter



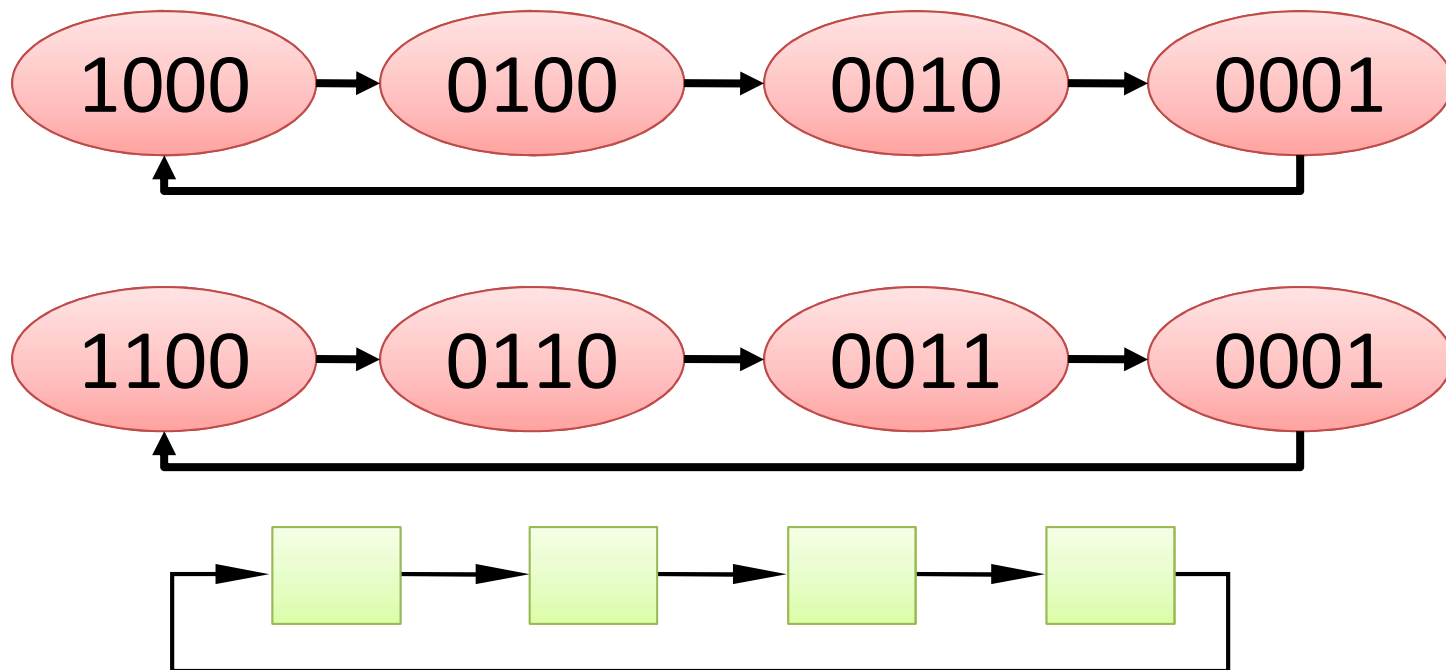
Sync: Down Binary Counter



Sync: Down Binary Counter



Counter Based on Shift Register



Design of Digital Wall Clock

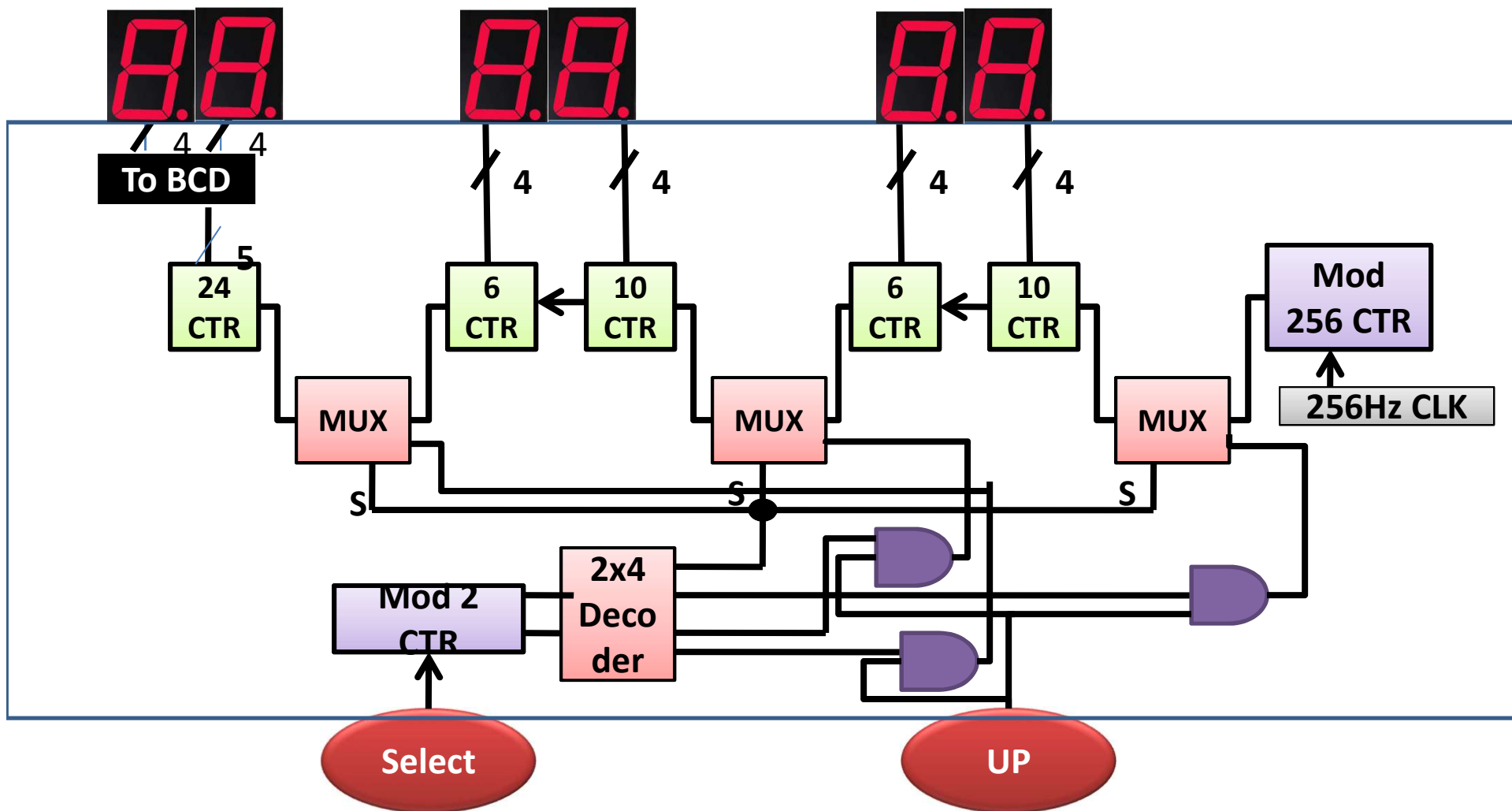
- Given 256Mhz Clock Quartz and other Digital components

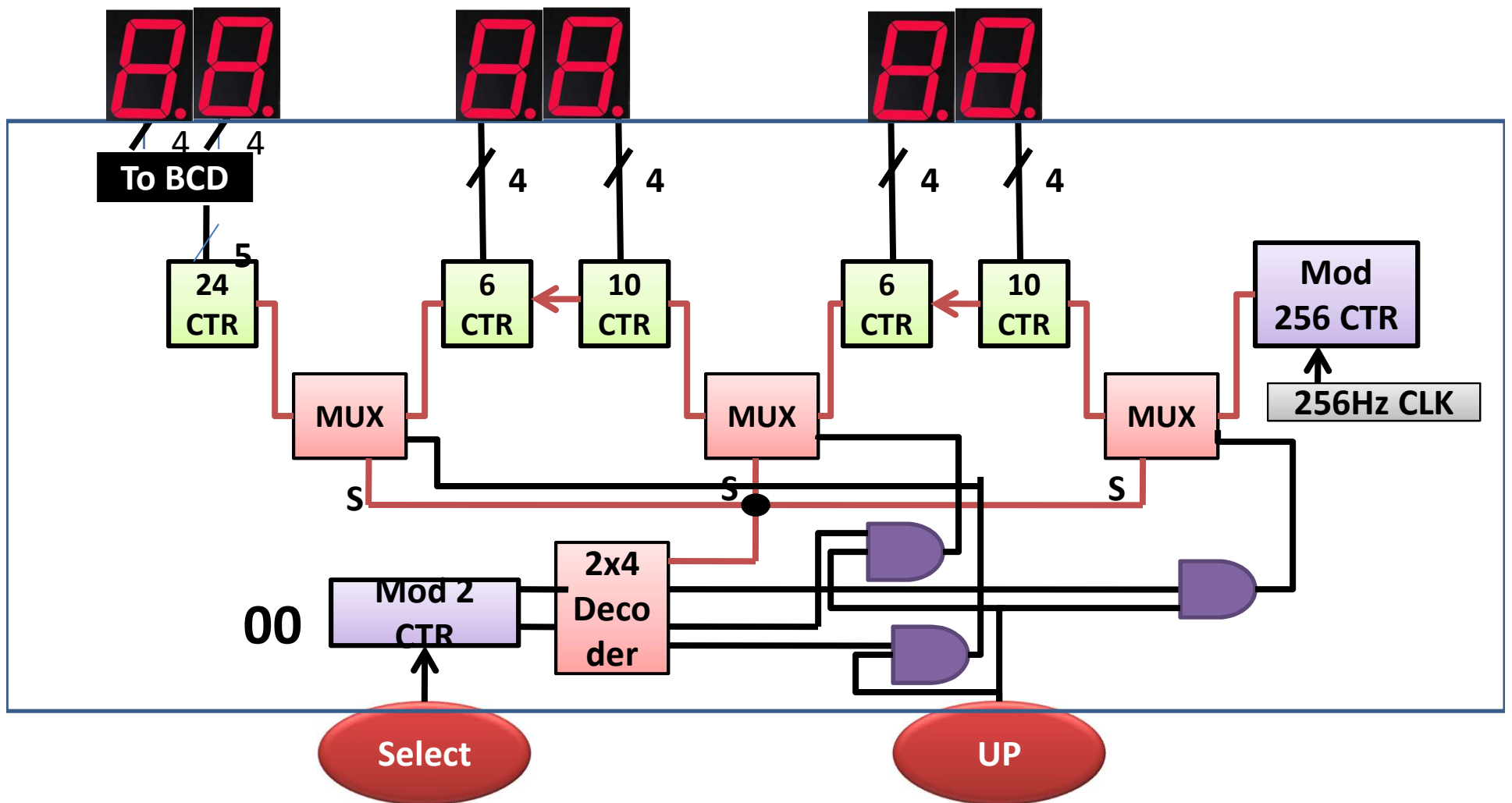


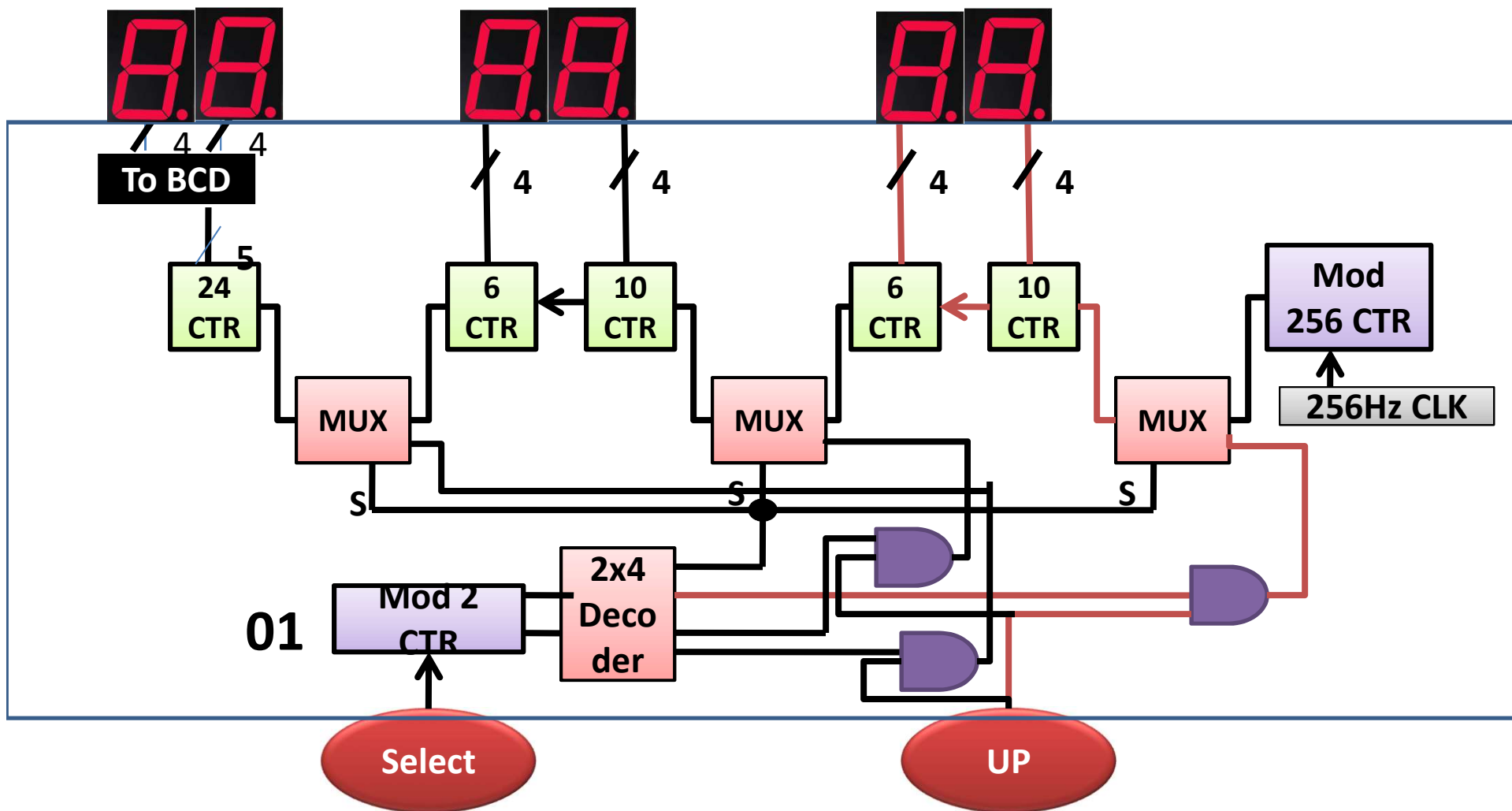
Design of Digital Wall Clock

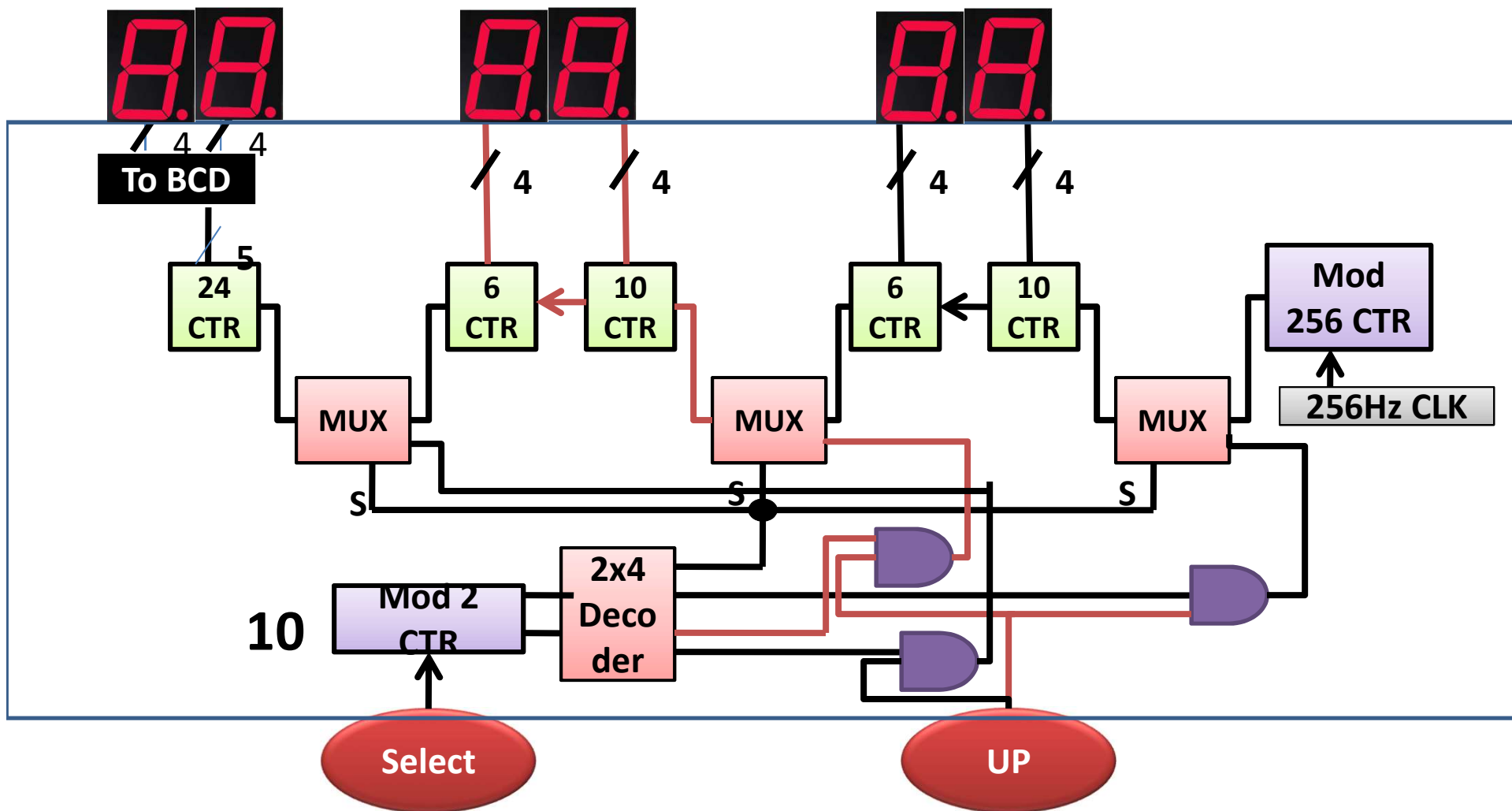
- Given 256Mhz Clock Quartz and other Digital components
- Design a Wall Clock
 - To display time : HH : MM :SS format
 - Should support Reset/Adjust of time u switch
 - Button 1: for select the Mod Ctr
 - Button 2: increasing select mod Ctr

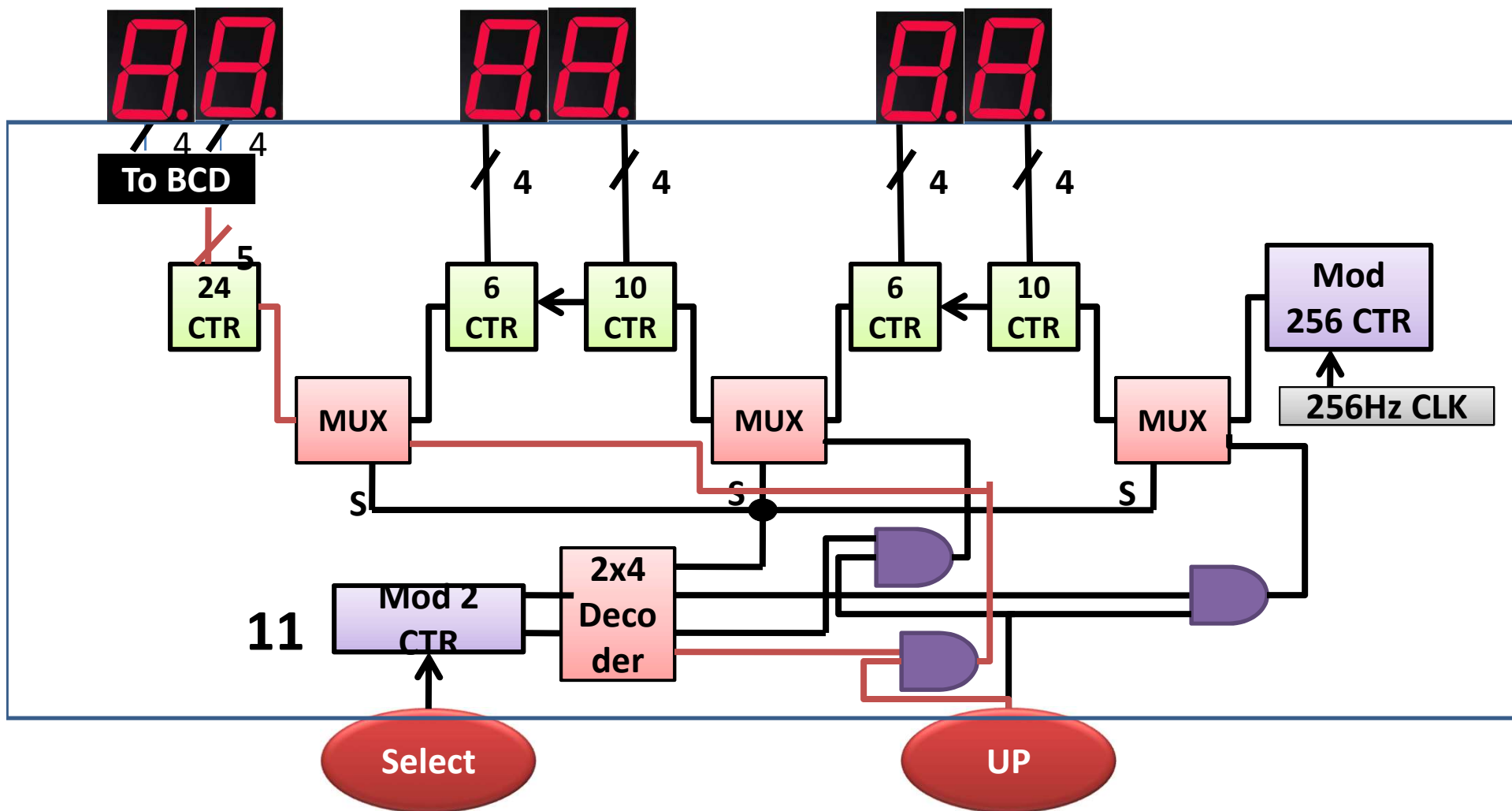












Thanks